Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

PIXEL STUDIO

Sistema de Gestión de un Estudio Fotográfico

Proyecto final: Diseño de Base de Datos

Asignatura: Bases de Datos
Profesor: Mtro. Roberto Elvira Enríquez
Alumno(a): Carlos Uriel Mora Contreras
Matrícula: 202131417
Fecha: Noviembre de 2026

Puebla, Puebla, México

Índice

1. Introducción 2

2. Planteamiento y justificación del problema 2

3. Diseño entidad–relación 2

4. Diccionario de datos 5

5. Normalización 8

6. Tablas, restricciones y CRUD 9

7. Procedimientos almacenados y funciones 10

8. Disparadores y vistas 11

9. Jobs 12

10. Índices, usuarios y permisos 14

11. Respaldos y notificaciones 14

12. Conclusión 15

13. Referencias 15

1 Introducción

Este proyecto desarrolla la base de datos de **Pixel Studio**, un estudio fotográfico ficticio que necesita controlar clientes, servicios, paquetes, reservaciones, sesiones, personal, equipo, pagos, fotografías y entregas. Se implementó en Microsoft SQL Server y se aplican los temas del curso: diseño E–R, normalización, tablas con restricciones, CRUD, procedimientos y funciones, disparadores y vistas, Jobs, índices, permisos y respaldos. Las capturas salen de la instancia donde se hicieron las pruebas.

2 Planteamiento y justificación del problema

Un estudio que lleva sus reservaciones en libretas o mensajes puede apartar dos sesiones al mismo fotógrafo, prestar un equipo ya asignado, perder el control de los pagos o no saber qué fotos se entregaron. Una base de datos relacional centraliza la información y hace que el propio sistema valide las reglas.

Objetivo general: diseñar e implementar en SQL Server una base de datos para gestionar las operaciones principales de un estudio fotográfico, con controles de integridad y herramientas de administración.

Reglas del negocio:

- Cada reservación pertenece a un cliente y a un paquete; cada paquete, a un servicio.
- Una reservación genera como máximo una sesión, con un fotógrafo principal.
- Una sesión puede usar varios equipos y un equipo puede usarse en varias sesiones.
- Los pagos se registran por reservación y no pueden sumar más que el precio pactado.
- Las fotografías pertenecen a una sesión y se agrupan en entregas.
- Los importes no son negativos, el pago es mayor a cero y la sesión termina después de empezar.

3 Diseño entidad–relación

El modelo tiene doce entidades: `Clientes`, `Servicios`, `Paquetes`, `Empleados`, `Equipos`, `Reservaciones`, `Sesiones`, `Pagos`, `AsignacionesEquipo`, `Fotografias`, `Entregas` y `DetalleEntrega`. Las dos últimas tablas de enlace resuelven relaciones N:M.

Entidad origen	Entidad relacionada	Cardinalidad
Clientes	Reservaciones	1:N
Servicios	Paquetes	1:N
Paquetes	Reservaciones	1:N
Reservaciones	Sesiones	1:0..1
Empleados	Sesiones	1:N
Reservaciones	Pagos	1:N
Sesiones	AsignacionesEquipo	1:N
Equipos	AsignacionesEquipo	1:N
Sesiones	Fotografias	1:N
Sesiones	Entregas	1:N
Entregas	DetalleEntrega	1:N
Fotografias	DetalleEntrega	1:N

Tabla 1: Relaciones principales.

El modelo se dividió en dos diagramas para que se lea bien; `Sesiones` aparece en ambos porque los une, pero es una sola tabla.

Modelo E–R A. Contratación y programación

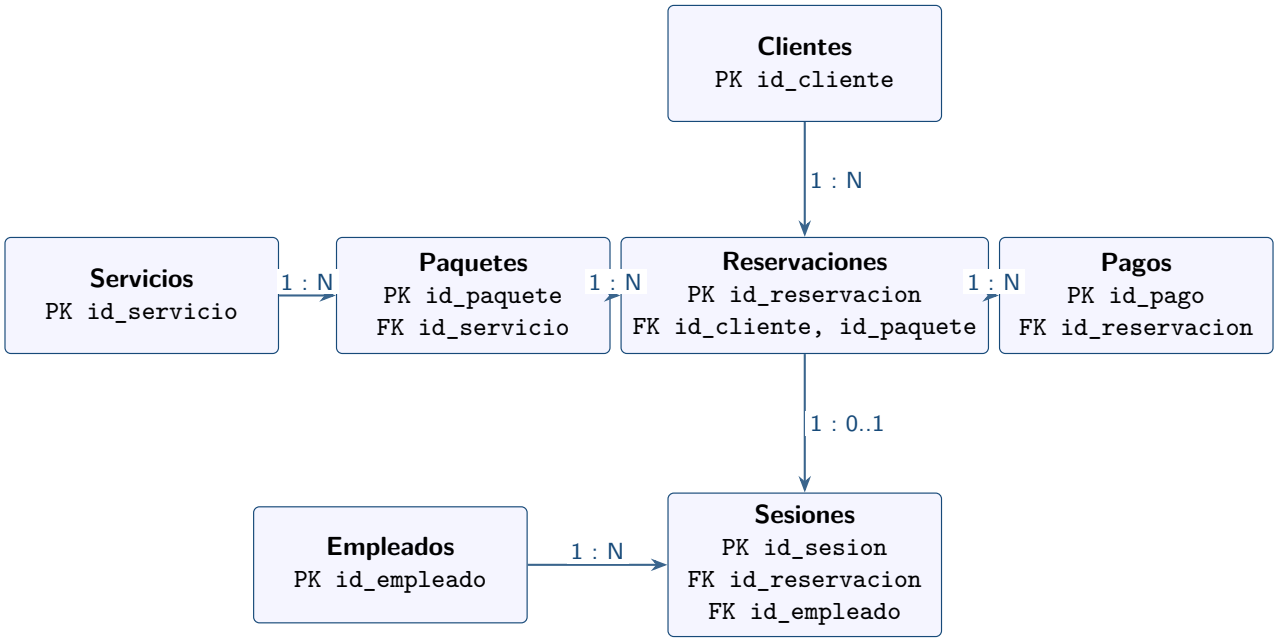


Figura 1: Diagrama E–R A: contratación, pagos y fotógrafo de la sesión.

Modelo E–R B. Equipos, fotografías y entregas

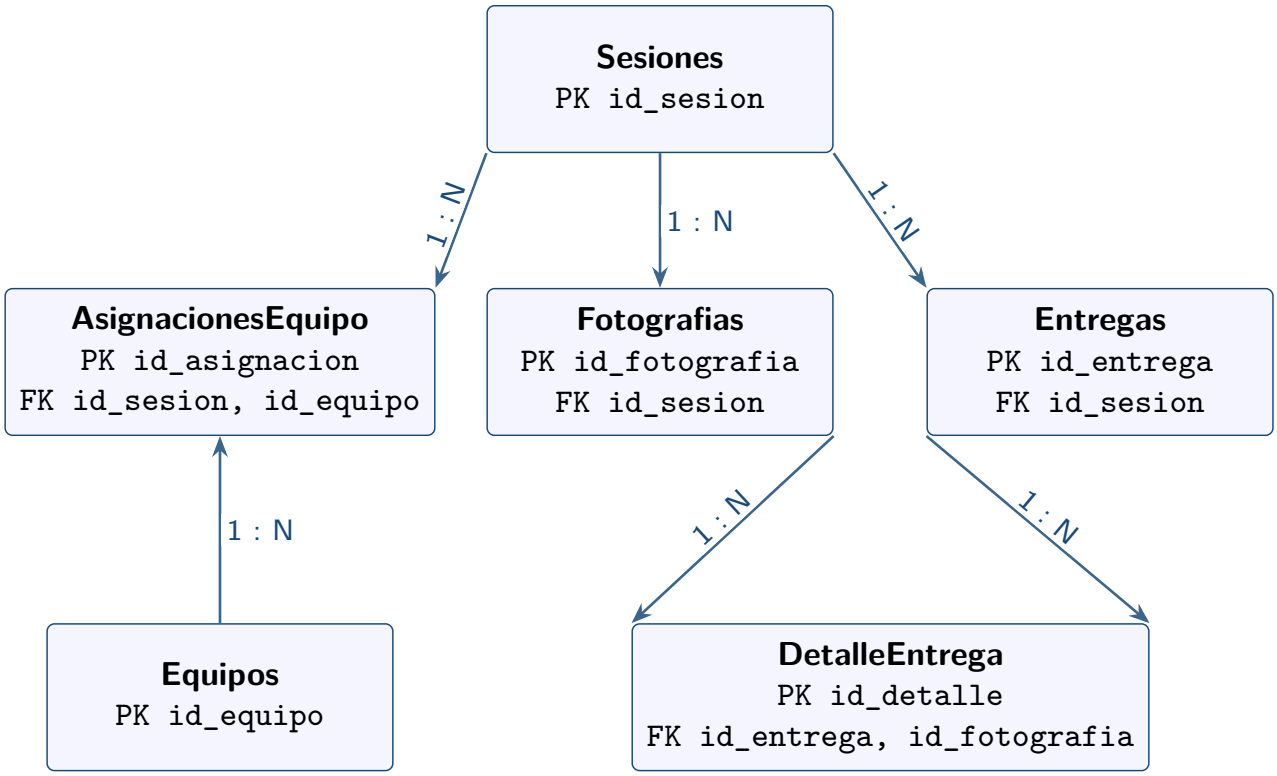


Figura 2: Diagrama E–R B: equipo, fotografías y entregas.



	Nombre_Tabla
1	AsignacionesEquipo
2	BitacoraJobs
3	Clientes
4	DetalleEntrega
5	Empleados
6	Entregas
7	Equipos
8	Fotografias
9	Pagos
10	Paquetes
11	Reservaciones
12	Servicios
13	Sesiones

Figura 3: Las doce tablas del modelo y `BitacoraJobs` en SQL Server.

4 Diccionario de datos

Obtenido del script `02-tablas/01_crear_bd_tablas.sql`. PK es clave primaria y FK, clave foránea.

Clientes

Atributo	Tipo SQL	Características
`id_cliente`	`INT`	PK, obligatorio
`nombre`	`NVARCHAR(80)`	Obligatorio
`apellido_paterno`	`NVARCHAR(80)`	Obligatorio
`apellido_materno`	`NVARCHAR(80)`	Opcional
`telefono`	`VARCHAR(20)`	Obligatorio
`correo`	`NVARCHAR(150)`	Opcional
`fecha_registro`	`DATETIME2(0)`	Obligatorio, predeterminado

Servicios

Atributo	Tipo SQL	Características
`id_servicio`	`INT`	PK, obligatorio
`nombre`	`NVARCHAR(100)`	Obligatorio
`descripcion`	`NVARCHAR(500)`	Opcional
`activo`	`BIT`	Obligatorio, predeterminado

Paquetes

Atributo	Tipo SQL	Características
`id_paquete`	`INT`	PK, obligatorio
`id_servicio`	`INT`	FK, obligatorio
`nombre`	`NVARCHAR(120)`	Obligatorio
`descripcion`	`NVARCHAR(500)`	Opcional
`precio`	`DECIMAL(10,2)`	Obligatorio, ≥ 0
`activo`	`BIT`	Obligatorio, predeterminado

Empleados

Atributo	Tipo SQL	Características
`id_empleado`	`INT`	PK, obligatorio
`nombre`	`NVARCHAR(80)`	Obligatorio
`apellido_paterno`	`NVARCHAR(80)`	Obligatorio
`apellido_materno`	`NVARCHAR(80)`	Opcional
`telefono`	`VARCHAR(20)`	Opcional

Atributo	Tipo SQL	Características
puesto	VARCHAR(30)	Obligatorio, lista permitida
fecha_contratacion	DATE	Obligatorio
activo	BIT	Obligatorio, predeterminado

Equipos

Atributo	Tipo SQL	Características
id_equipo	INT	PK, obligatorio
nombre	NVARCHAR(120)	Obligatorio
numero_serie	NVARCHAR(100)	Opcional
tipo	VARCHAR(30)	Obligatorio
estado	VARCHAR(20)	Obligatorio, predeterminado, lista permitida

Reservaciones

Atributo	Tipo SQL	Características
id_reservacion	INT	PK, obligatorio
id_cliente	INT	FK, obligatorio
id_paquete	INT	FK, obligatorio
fecha_reservacion	DATE	Obligatorio
hora_reservacion	TIME(0)	Obligatorio
precio_pactado	DECIMAL(10,2)	Obligatorio, ≥ 0
estado	VARCHAR(20)	Obligatorio, predeterminado, lista permitida
observaciones	NVARCHAR(250)	Opcional

Sesiones

Atributo	Tipo SQL	Características
id_sesion	INT	PK, obligatorio
id_reservacion	INT	FK, obligatorio, único
id_empleado	INT	FK, obligatorio
fecha_inicio	DATETIME2(0)	Obligatorio
fecha_fin	DATETIME2(0)	Obligatorio, mayor que el inicio
estado	VARCHAR(20)	Obligatorio, predeterminado, lista permitida

Pagos

Atributo	Tipo SQL	Características
id_pago	INT	PK, obligatorio
id_reservacion	INT	FK, obligatorio
monto	DECIMAL(10,2)	Obligatorio, > 0
fecha_pago	DATETIME2(0)	Obligatorio, predeterminado
metodo_pago	VARCHAR(20)	Obligatorio, lista permitida
estado	VARCHAR(20)	Obligatorio, predeterminado, lista permitida

AsignacionesEquipo

Atributo	Tipo SQL	Características
id_asignacion	INT	PK, obligatorio
id_sesion	INT	FK, obligatorio
id_equipo	INT	FK, obligatorio

Fotografias

Atributo	Tipo SQL	Características
id_fotografia	INT	PK, obligatorio
id_sesion	INT	FK, obligatorio
nombre_archivo	NVARCHAR(200)	Obligatorio
ruta_archivo	NVARCHAR(500)	Obligatorio
fecha_captura	DATETIME2(0)	Opcional
estado	VARCHAR(20)	Obligatorio, predeterminado, lista permitida

Entregas

Atributo	Tipo SQL	Características
id_entrega	INT	PK, obligatorio
id_sesion	INT	FK, obligatorio
fecha_entrega	DATETIME2(0)	Obligatorio, predeterminado
metodo_entrega	VARCHAR(20)	Obligatorio, lista permitida
estado	VARCHAR(20)	Obligatorio, predeterminado, lista permitida

DetalleEntrega

Atributo	Tipo SQL	Características
`id_detalle`	`INT`	PK, obligatorio
`id_entrega`	`INT`	FK, obligatorio
`id_fotografia`	`INT`	FK, obligatorio

5 Normalización

Una hoja única con cliente, teléfono, paquete, servicio y columnas como `equipo1`, `equipo2`, `foto1`, `foto2` repite datos (el teléfono de un cliente en cada reservación), provoca anomalías al actualizar, insertar o borrar, y limita cuántos equipos o fotos caben.

1FN. Cada celda tiene un solo valor: los equipos de una sesión pasan a `AsignacionesEquipo` y las fotos entregadas a `DetalleEntrega`, un renglón por cada una.

2FN. En la relación sesión–equipo, la clave lógica es `(id_sesion, id_equipo)`. El nombre y la serie del equipo dependen solo de `id_equipo`, y las fechas solo de `id_sesion`; por eso viven en `Equipos` y `Sesiones`, y la tabla intermedia solo guarda las claves.

3FN. `Reservaciones` guarda solo las claves de cliente y paquete; el nombre del cliente, del paquete y del servicio se consultan en sus tablas, así se evitan dependencias transitivas. `precio_pactado` sí se guarda en `Reservaciones` a propósito: es el precio acordado en ese momento y no debe cambiar si el paquete sube de precio.

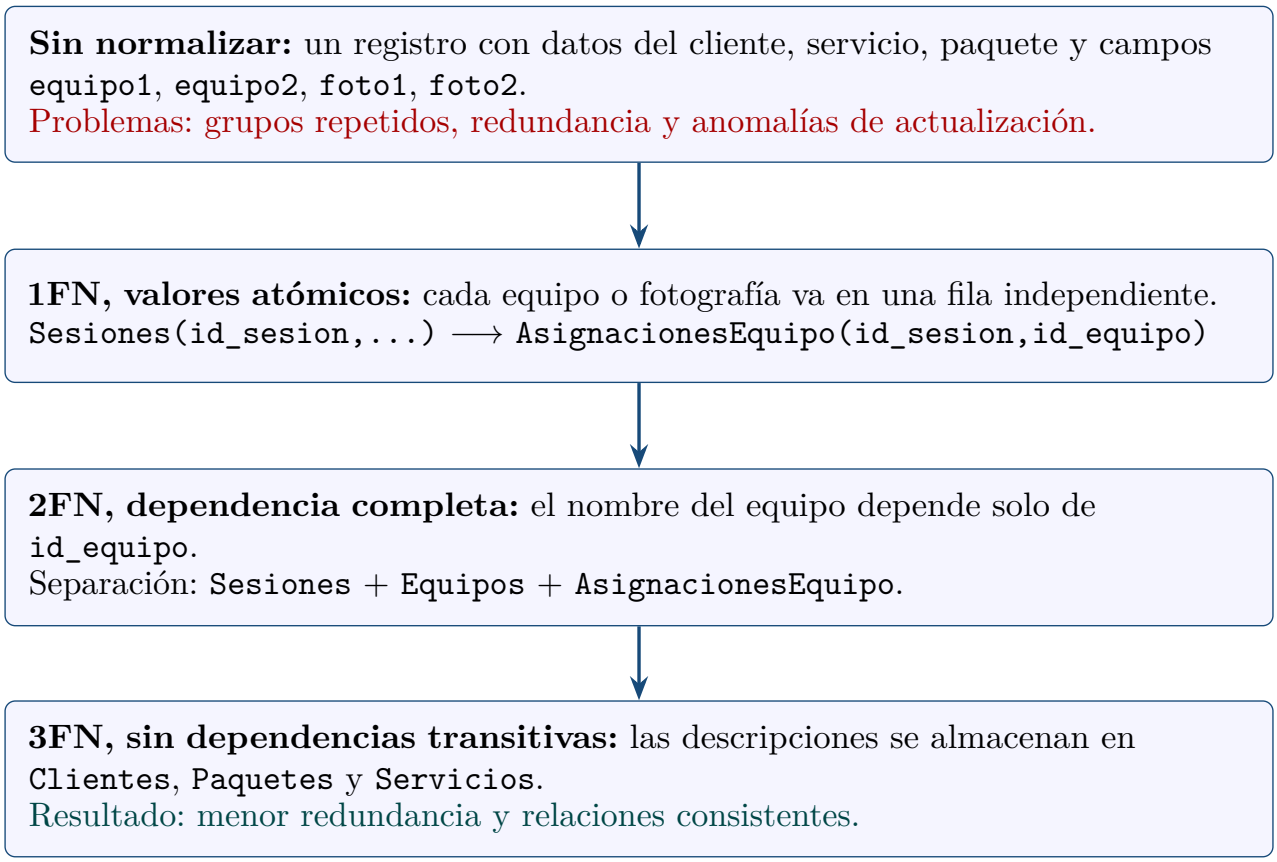


Figura 4: Proceso de normalización hasta la 3FN.

6 Tablas, restricciones y CRUD

El script `01_crear_bd_tablas.sql` crea las doce tablas con `IDENTITY`, `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL`, `DEFAULT` y `CHECK`. Los `CHECK` validan precios no negativos, pagos mayores a cero, listas de estados y que `fecha_fin >fecha_inicio` en las sesiones. Un ejemplo:

```
CONSTRAINT UQ_Sesiones_Reservacion UNIQUE (id_reservacion),
CONSTRAINT CK_Sesiones_Fechas CHECK (fecha_fin > fecha_inicio)
```

	Tabla	Restriccion	Condicion
1	Empleados	CK_Empleados_Puesto	([puesto]='Administrativo' OR [puesto]='Editor' OR [p...
2	Entregas	CK_Entregas_Estado	([estado]='Cancelada' OR [estado]='Entregada' OR [...
3	Entregas	CK_Entregas_Metodo	([metodo_entrega]='Fisica' OR [metodo_entrega]='Di...
4	Equipos	CK_Equipos_Estado	([estado]='Baja' OR [estado]='Mantenimiento' OR [e...
5	Fotografias	CK_Fotografias_Estado	([estado]='Seleccionada' OR [estado]='Editada' OR ...
6	Pagos	CK_Pagos_Estado	([estado]='Reembolsado' OR [estado]='Aplicado')
7	Pagos	CK_Pagos_Metodo	([metodo_pago]='Tarjeta' OR [metodo_pago]='Trans...
8	Pagos	CK_Pagos_Monto	([monto]>(0))
9	Paquetes	CK_Paquetes_Precio	([precio]>=(0))
10	Reservaciones	CK_Reservaciones_Estado	([estado]='Completada' OR [estado]='Cancelada' O...
11	Reservaciones	CK_Reservaciones_Precio	([precio_pactado]>=(0))
12	Sesiones	CK_Sesiones_Estado	([estado]='Cancelada' OR [estado]='Realizada' OR [...
13	Sesiones	CK_Sesiones_Fechas	([fecha_fin]>[fecha_inicio])

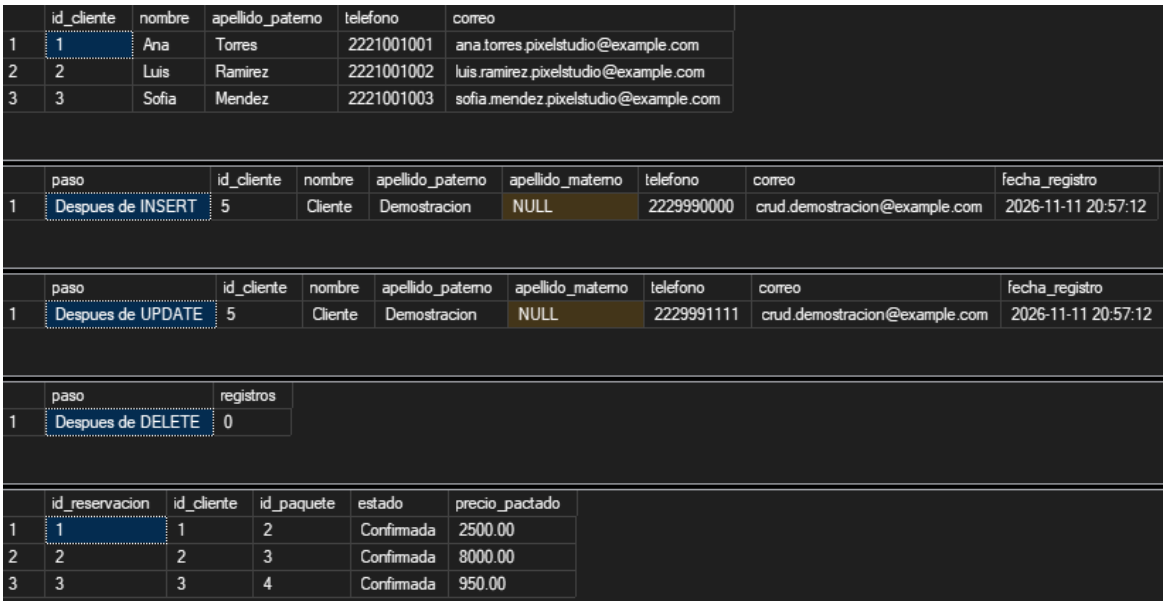
Figura 5: Restricciones `CHECK` registradas en la base.

Se cargaron datos ficticios (tres clientes, cuatro paquetes y tres reservaciones) y se probó el CRUD con un cliente de demostración: se insertó, se cambió su teléfono y se eliminó, y la consulta final devuelve 0 registros.

	id_cliente	nombre	apellido_paterno	apellido_materno	telefono	correo	fecha_registro
1	1	Ana	Torres	Lopez	2221001001	ana.torres.pixelstudio@example.com	2026-11-09 20:59:21
2	2	Luis	Ramirez	NULL	2221001002	luis.ramirez.pixelstudio@example.com	2026-11-09 20:59:21
3	3	Sofia	Mendez	Garcia	2221001003	sofia.mendez.pixelstudio@example.com	2026-11-09 20:59:21

	id_paquete	id_servicio	nombre	descripcion	precio	activo
1	1	1	Basico	10 fotografias editadas	1200.00	1
2	2	1	Premium	25 fotografias y album	2500.00	1
3	3	2	Esencial	Cobertura de ceremonia	8000.00	1
4	4	3	Individual	8 retratos editados	950.00	1

Figura 6: Clientes y paquetes de prueba.



	id_cliente	nombre	apellido_paterno	telefono	correo
1	1	Ana	Torres	2221001001	ana.torres.pixelstudio@example.com
2	2	Luis	Ramirez	2221001002	luis.ramirez.pixelstudio@example.com
3	3	Sofia	Mendez	2221001003	sofia.mendez.pixelstudio@example.com

	paso	id_cliente	nombre	apellido_paterno	apellido_materno	telefono	correo	fecha_registro
1	Despues de INSERT	5	Cliente	Demostracion	NULL	2229990000	crud.demostracion@example.com	2026-11-11 20:57:12

	paso	id_cliente	nombre	apellido_paterno	apellido_materno	telefono	correo	fecha_registro
1	Despues de UPDATE	5	Cliente	Demostracion	NULL	2229991111	crud.demostracion@example.com	2026-11-11 20:57:12

	paso	registros
1	Despues de DELETE	0

	id_reservacion	id_cliente	id_paquete	estado	precio_pactado
1	1	1	2	Confirmada	2500.00
2	2	2	3	Confirmada	8000.00
3	3	3	4	Confirmada	950.00

Figura 7: Resultados de INSERT, UPDATE y DELETE.

También se hicieron consultas con `INNER JOIN` y `LEFT JOIN`, y resúmenes con `GROUP BY`, `SUM` y `COUNT`.

	nombre	apellido_paterno	paquete	fecha_reservacion	estado
1	Ana	Torres	Premium	2026-11-14	Confirmada
2	Luis	Ramirez	Esencial	2026-11-15	Confirmada
3	Sofia	Mendez	Individual	2026-11-14	Confirmada

Figura 8: Consulta `JOIN` entre clientes, reservaciones y paquetes.

7 Procedimientos almacenados y funciones

Se crearon los procedimientos `sp_CrearReservacion`, `sp_ProgramarSesion`, `sp_RegistrarPago` y `usp_RespaldarPixelStudio`, y las funciones `fn_SaldoReservacion` (escalar) y `fn_ResumenReservacion` (de tabla en línea). Los procedimientos concentran las operaciones que modifican datos con sus validaciones; las funciones se pueden usar dentro de un `SELECT`.

	Nombre	Tipo
1	fn_ResumenReservacion	SQL_INLINE_TABLE_VALUED_FUNCTION
2	fn_SaldoReservacion	SQL_SCALAR_FUNCTION
3	sp_CrearReservacion	SQL_STORED_PROCEDURE
4	sp_ProgramarSesion	SQL_STORED_PROCEDURE
5	sp_RegistrarPago	SQL_STORED_PROCEDURE
6	usp_RespaldarPixelStudio	SQL_STORED_PROCEDURE

Figura 9: Procedimientos y funciones registrados en SQL Server.

El saldo es el precio pactado menos los pagos aplicados: la reservación 1 queda con saldo de 2000, la 2 con 6000 y la 3 con 0.

	id_reservacion	precio_pactado	saldo_pendiente
1	1	2500.00	2000.00
2	2	8000.00	6000.00
3	3	950.00	0.00

Figura 10: Resultado de `fn_SaldoReservacion`.

8 Disparadores y vistas

Se crearon cuatro disparadores: `trg_Pagos_NoExcederTotal` (los pagos no superan el precio pactado), `trg_DetalleEntrega_MismaSesion` (una foto solo se entrega en su sesión), `trg_Sesiones_ValidarAgenda` (evita empalmes del fotógrafo) y `trg_AsignacionesEquipo_Disponibilidad` (revisa la disponibilidad del equipo). Las vistas son `vw_EstadoCuentaReservaciones`, `vw_AgendaFotografos`, `vw_InventarioEquipos` y `vw_ResumenEntregas`.

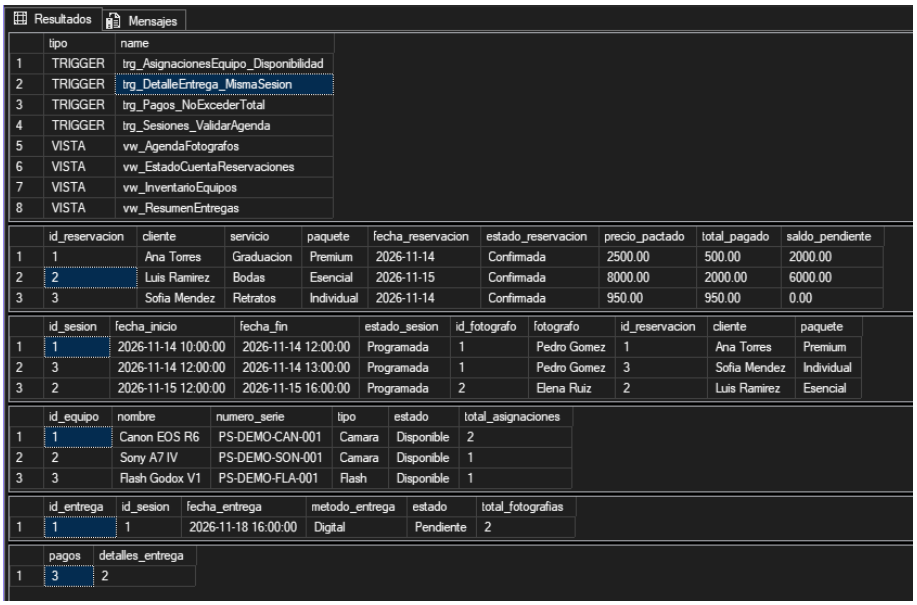


Figura 11: Cuatro disparadores, cuatro vistas y sus resultados.

En las pruebas negativas el sistema rechazó un pago que excedía el precio pactado (error 52007) y una foto de otra sesión (error 52001), sin agregar registros.

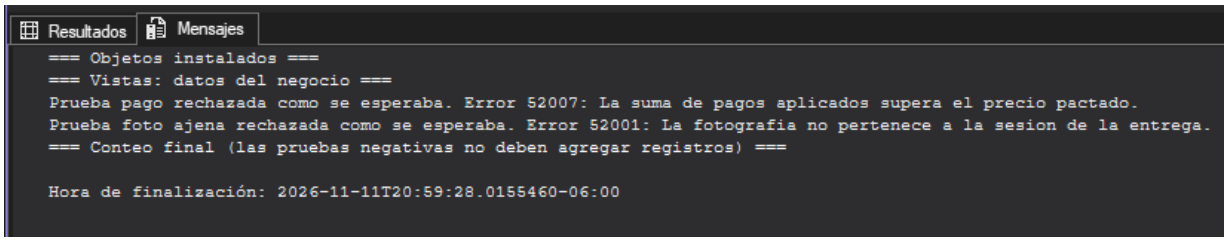


Figura 12: Rechazo controlado de operaciones inválidas.

9 Jobs

Se configuró SQL Server Agent y se creó `JOB_PixelStudio_Mantenimiento`, cuyo paso inserta un registro en `BitacoraJobs`. Se ejecutó manualmente con éxito y la bitácora registró la corrida. También se creó `JOB_PixelStudio_RespaldoDiario`, programado todos los días a las 11:30 p. m.

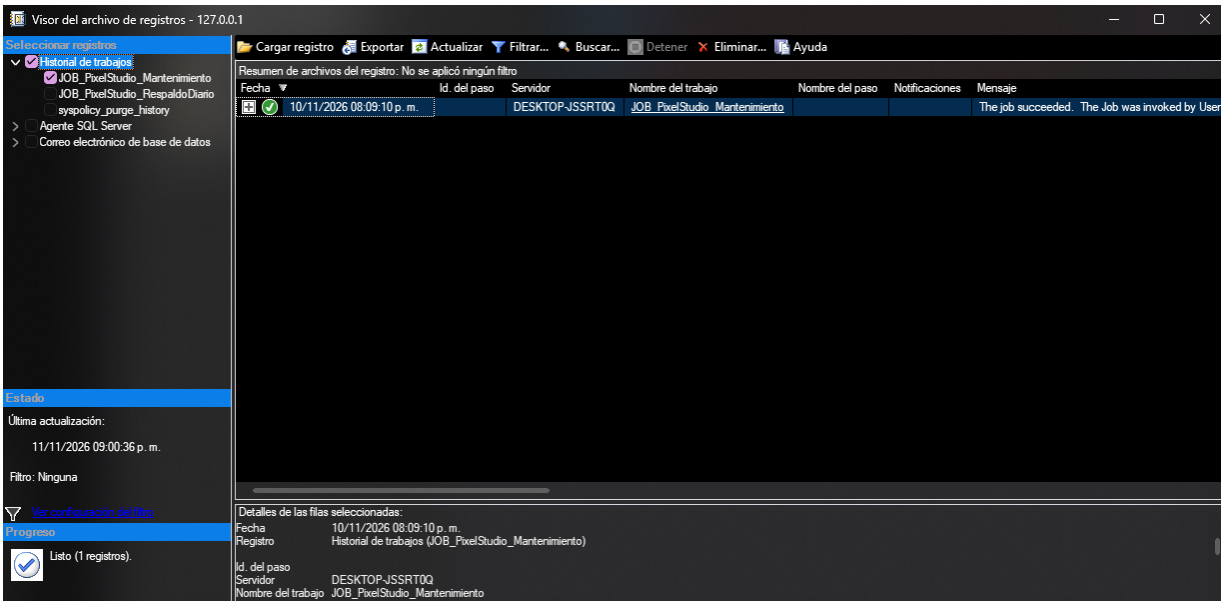


Figura 13: Historial de ejecución exitosa del Job de mantenimiento.

	id_ejecucion	nombre_job	fecha_ejecucion	resultado
1	1	JOB_PixelStudio_Mantenimiento	2026-11-10 20:09:10.4026164	Ejecutado correctamente

Figura 14: Registro generado en `BitacoraJobs`.

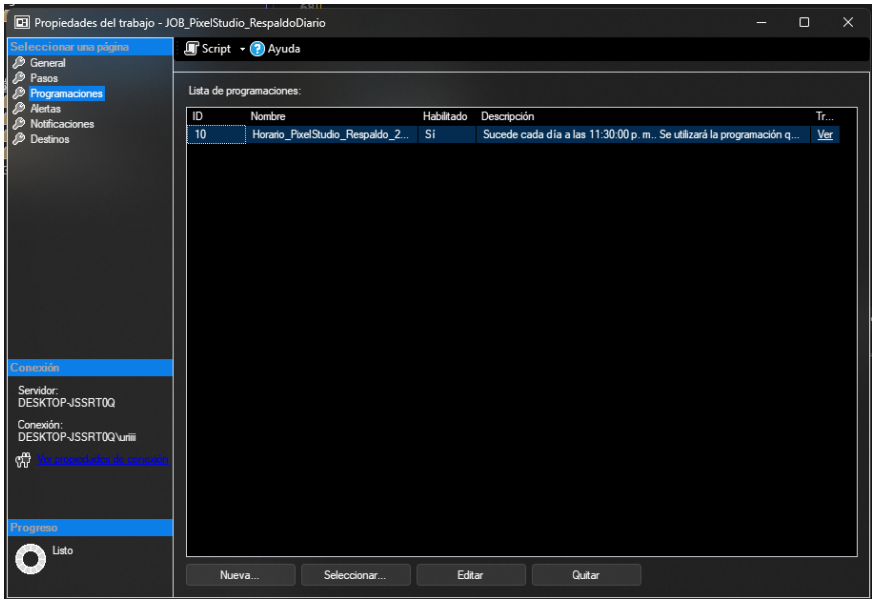


Figura 15: Programación del Job de respaldo diario.

10 Índices, usuarios y permisos

Como SQL Server no indexa las claves foráneas por sí solo, se crearon cinco índices no agrupados: `IX_Reservaciones_Cliente_Fecha`, `IX_Sesiones_Fotografo_Inicio`, `IX_Pagos_Reservacion_Estado`, `IX_Fotografias_Sesion` e `IX_AsignacionesEquipo_Equipo`.

	Tabla	Indice	Tipo
1	AsignacionesEquipo	IX_AsignacionesEquipo_Equipo	NONCLUSTERED
2	Fotografias	IX_Fotografias_Sesion	NONCLUSTERED
3	Pagos	IX_Pagos_Reservacion_Estado	NONCLUSTERED
4	Reservaciones	IX_Reservaciones_Cliente_Fecha	NONCLUSTERED
5	Sesiones	IX_Sesiones_Fotografo_Inicio	NONCLUSTERED

Figura 16: Índices creados.

En seguridad se creó el rol `rol_pixelstudio_consulta`, con lectura sobre el esquema `dbo`, y el usuario `usuario_pixelstudio_demo` dentro de ese rol, siguiendo el principio de mínimo privilegio: puede consultar, pero no modificar datos.

	Rol	Usuario
1	rol_pixelstudio_consulta	usuario_pixelstudio_demo

Figura 17: Rol y usuario de consulta.

11 Respaldos y notificaciones

El procedimiento `usp_RespaldarPixelStudio` genera un respaldo completo de la base en un archivo `.bak`, y el script `02_verificar_respaldo.sql` consulta el historial. Se comprobó el respaldo completo (tipo D) del 11 de noviembre de 2026 y la ruta del archivo. `RESTORE VERIFYONLY` valida que el archivo sea legible, pero no sustituye una restauración completa de prueba.

	BaseDatos	Inicio	Fin	Tipo
1	PixelStudio	2026-11-11 20:23:26.000	2026-11-11 20:23:26.000	D

Figura 18: Respaldo completo de `PixelStudio`.

	BaseDatos	FechaRespaldo	RutaArchivo
1	PixelStudio	2026-11-11 20:23:26.000	C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\Backup\PixelStudio_20261111_202326_E786C6BE.bak

Figura 19: Ruta del archivo `.bak` en el historial.

Notificaciones: se revisó Database Mail y los operadores del Agente, pero no se logró configurar la conexión SMTP, por lo que el envío de correos quedó **pendiente**. Para completarlo se crearía un perfil de Database Mail, un operador y una notificación de fallo en cada Job.

12 Conclusión

Con Pixel Studio se recorrió el proceso completo de una base de datos: análisis del problema, modelo E–R, normalización, tablas con restricciones y objetos de programación, además de Jobs, permisos y respaldos. La normalización evitó datos repetidos, y los disparadores permitieron validar reglas que un `CHECK` no puede, como que los pagos no superen el precio pactado. Quedan pendientes las notificaciones por correo, comprobar la ejecución automática del respaldo diario y probar una restauración completa.

13 Referencias

- Elvira Enríquez, R. (2026). *Presentaciones BD 1–BD 6*. Material de clase, FCC, BUAP.
- Elmasri, R. y Navathe, S. (2016). *Fundamentals of Database Systems*. Pearson.
- Silberschatz, A., Korth, H. y Sudarshan, S. (2010). *Database System Concepts*. McGraw-Hill.
- Microsoft Learn. *Documentación de SQL Server*. https://learn.microsoft.com/sql/